

Problem and Goal

- Parallel Machine Scheduling
- Jobs, machines, precedences, setup times and resources
- Goal: assign every job to a machine and a start time
- Objective: minimize total tardiness + makespan
- All Topic A constraints are treated as hard constraints

Algorithm Description

- Metaheuristic: Simulated Annealing
- Single-solution local search method
- Starts from one current solution
- Generates one neighbour at a time
- Accepts better solutions
- Sometimes accepts worse solutions to escape local optima
- Temperature decreases during the search

Representation and Decoder

- Representation: priority list of jobs and selected machine for each job
- The decoder converts this representation into real start times
- Only jobs whose predecessors are finished can be scheduled
- The decoder checks machine eligibility, setup times, precedences and resources
- The final solution is checked with the official validator

Neighbourhood and Acceptance

- Swap two jobs in the priority list
- Remove and reinsert one job
- Change one job to another eligible machine
- Better neighbours are accepted
- Worse neighbours can be accepted depending on cost difference and temperature

Parameter Variants

- short: 5000 iterations, temperature 250, cooling 0.9990
- default: 10000 iterations, temperature 500, cooling 0.9995
- hot: 10000 iterations, temperature 1000, cooling 0.9997
- Each variant is run 5 times with different random seeds
- Results are best found values, not optimality proofs

Benchmark Results

- Average / best / standard deviation over 5 runs:
- j10_m3_r0_1: avg 2697.0, best 2697, std 0.0
- j10_m3_r0_2: avg 6050.0, best 6050, std 0.0
- j10_m3_r1_5: avg 3380.0, best 3380, std 0.0
- j10_m3_r2_3: avg 4009.0, best 4009, std 0.0
- j10_m4_r1_4: avg 14602.0, best 14602, std 0.0
- Full data is stored in results_assignment2.csv

Conclusions and Lessons Learned

- The decoder is the key part of the implementation
- Resource constraints cannot be ignored
- Simulated Annealing gives feasible best-found solutions
- It does not prove optimality
- More iterations can improve results, but also increase runtime

How to Run

- Single instance:
- `python3 metaheuristic_solver.py --instance instances/PSSAI_PMS_j10_m3_r1_5.json --output results/PSSAI_PMS_j10_m3_r1_5.solution.json --seed 1 --iterations 10000`
- Experiments:
- `python3 run_experiments.py --instances "instances/PSSAI_PMS_j10_*.json" --runs 5 --iterations 10000`